

Análise Empírica e Comparativa de Desempenho de Algoritmos de Ordenação

Universidade Federal do Tocantins (UFT) | Ciência da Computação

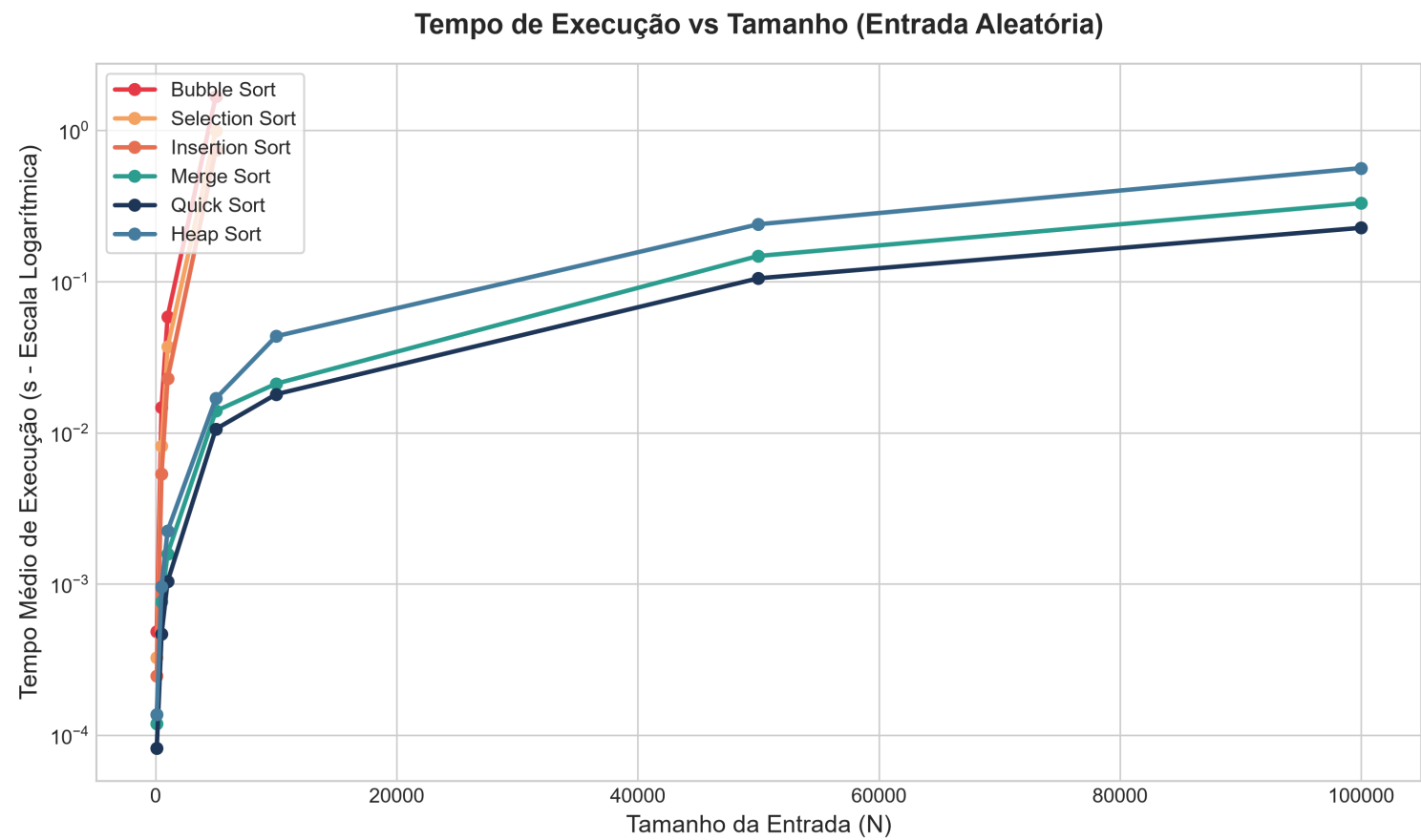
Trabalho Prático - Análise e Projeto de Algoritmos

Objetivos e Algoritmos Avaliados

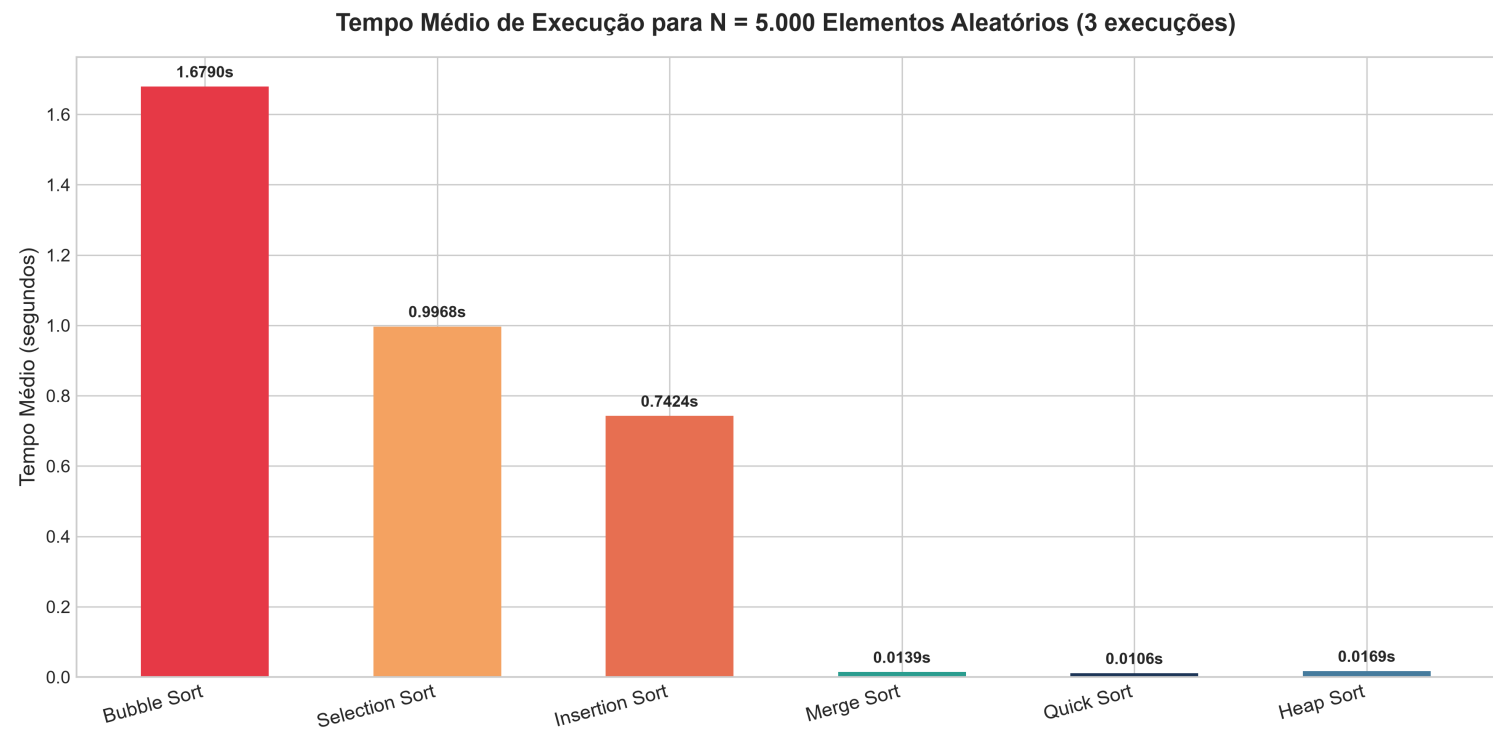
- Objetivo: Analisar experimentalmente o comportamento de 6 algoritmos de ordenação e relacionar com a complexidade assintótica.
- Algoritmos Analisados:
 1. Bubble Sort (Trocadas adjacentes com parada antecipada)
 2. Selection Sort (Seleção iterativa do mínimo)
 3. Insertion Sort (Inserção direta e incremental)
 4. Merge Sort (Divisão e Conquista estável)
 5. Quick Sort (Partição por Mediana de Três)
 6. Heap Sort (Max-Heap binário in-place)
- Métricas: Tempo de CPU (`perf_counter`), Comparações, Trocas/Deslocamentos e Validação.

Tabela Teórica de Complexidades

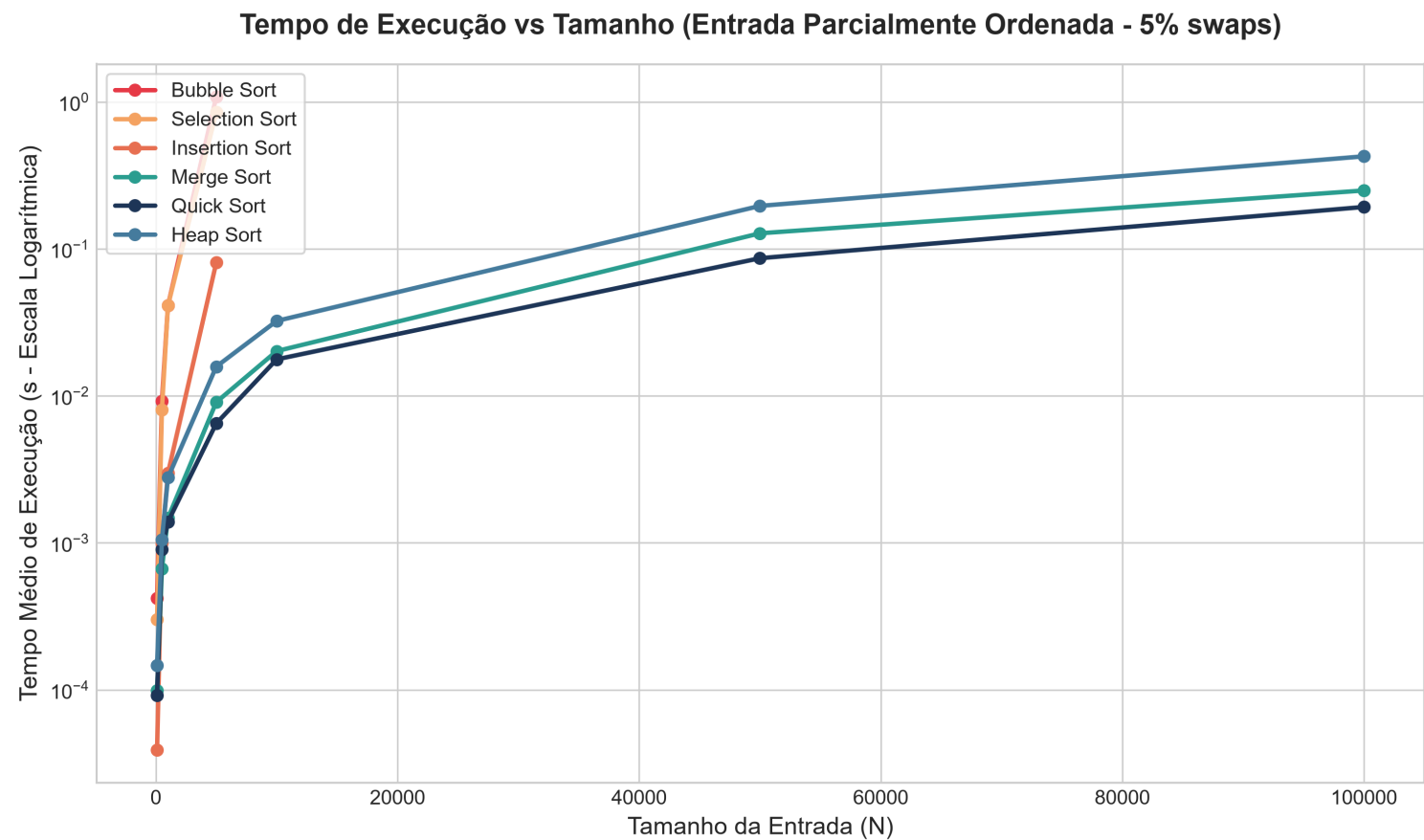
Algoritmo	Melhor Caso	Caso Médio	Pior Caso	Espaço	Estável?	Estratégia
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim	Trocas adjacentes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Não	Mínimo iterativo
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim	Inserção direta
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sim	Divisão e Conquista
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Não	Particionamento (Med3)
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Não	Max-Heap Binário

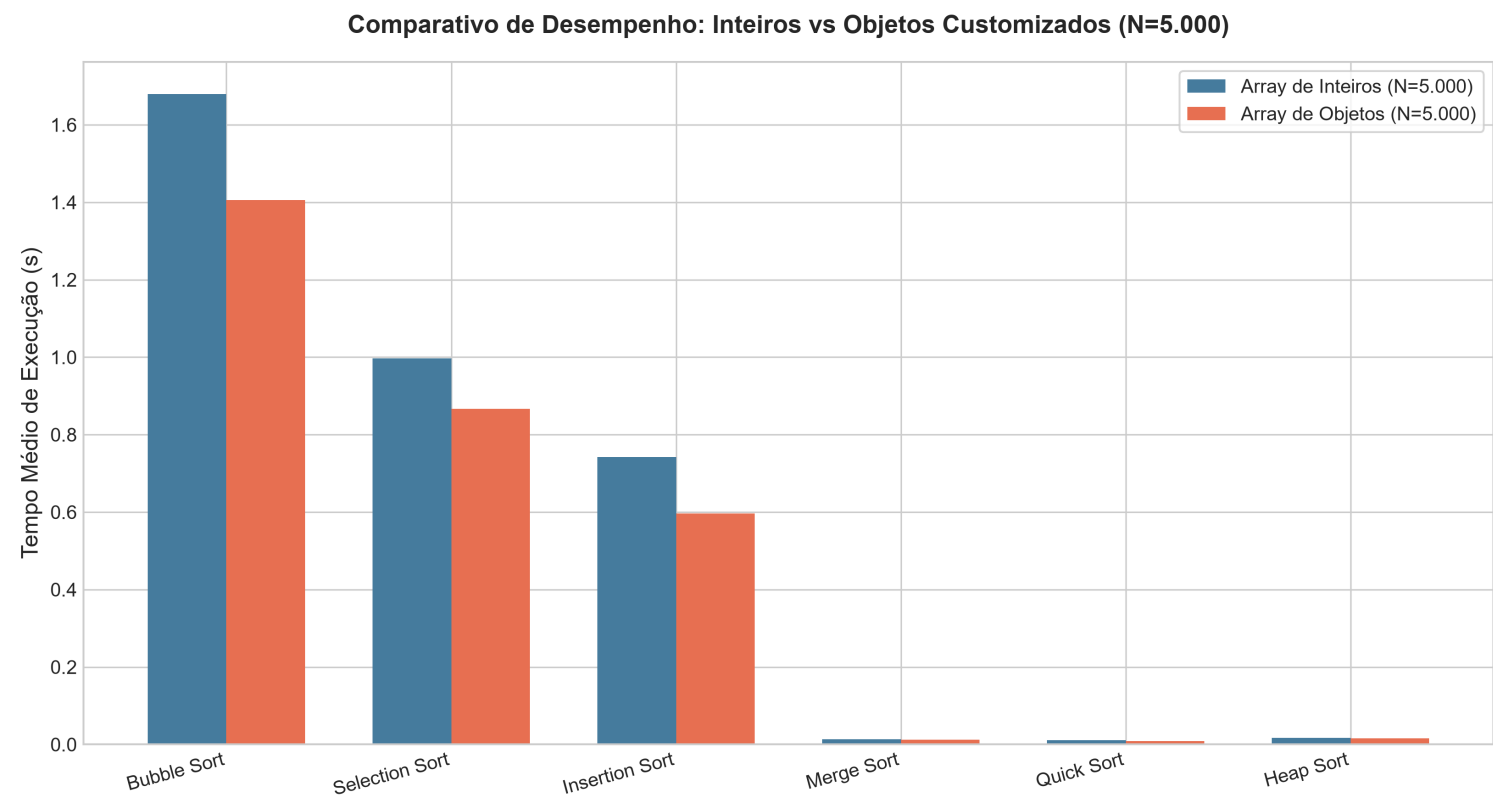


Comparativo Direto: N = 5.000 Elementos Aleatórios



Comportamento em Dados Quase Ordenados (5% desordem)





Conclusões e Recomendações Práticas

- Quick Sort: Algoritmo mais rápido para entradas aleatórias gerais devido ao cache e baixa constante.
- Insertion Sort: Excelente escolha para arrays pequenos ($N < 50$) e dados quase ordenados (complexidade quase linear $O(n)$).
- Merge Sort: Indispensável quando a estabilidade da ordenação é um requisito mandatório.
- Heap Sort: Garantia rigorosa de tempo $O(n \log n)$ com espaço de memória estritamente $O(1)$.
- Conclusão Teórica: Os experimentos práticos validaram perfeitamente as cotas assintóticas Big-O.